

UNIVERSITE — FACULTE DES SCIENCES ET TECHNIQUES

Licence Professionnelle Informatique

RAPPORT DE PROJET DE FIN D'ETUDES

FreeNest

Conception et Developpement d'une Marketplace Freelance Full-Stack avec Intelligence
Artificielle Locale

Realise par : Ayoub Elmernissi

Email : ayoubelmerniss55@gmail.com

Formation : Licence Professionnelle Informatique

Annee : 2025 - 2026

Intitule : FreeNest — Marketplace Freelance Full-Stack (PFE)

Stack : Laravel 12 | React 19 | MySQL 8 | Stripe | Docker | Ollama

Date de depot : Juin 2026

Remerciements

Je tiens a exprimer ma profonde gratitude envers toutes les personnes qui ont contribue, de pres ou de loin, a la realisation de ce projet de fin d'etudes.

Mes remerciements vont en premier lieu a mon encadrant pedagogique pour ses conseils eclaires, sa disponibilite constante et sa rigueur tout au long de l'avancement du projet. Ses orientations m'ont permis de maintenir un cap technique et methodologique solide dans les moments de doute.

Je remercie egalement l'ensemble du corps enseignant de ma formation pour la qualite des enseignements dispenses, qui ont constitue le socle de competences mobilise dans ce travail.

Un grand merci a ma famille pour son soutien moral, sa patience et ses encouragements constants durant cette periode de travail intensif.

Enfin, je tiens a remercier la communaute open-source internationale dont les travaux — Laravel, React, Stripe, Docker, Ollama, PlantUML — ont rendu ce projet possible et m'ont permis de travailler avec les outils de l'industrie 2026.

Resume

FreeNest est une marketplace freelance full-stack de niveau production, developpee dans le cadre d'un Projet de Fin d'Etudes. La plateforme met en relation des freelancers independants et des clients entreprises au travers d'un ecosysteme technologique complet et moderne.

Sur le plan fonctionnel, FreeNest couvre l'integralite du cycle de vie d'une mission freelance : inscription avec onboarding guide, authentication multi-facteurs (2FA TOTP), verification d'identite KYC, publication et recherche d'offres, systeme de propositions avec

generation par IA locale, gestion complete de contrats (jalons, fichiers, suivi du temps, extensions, PDF), paiements securises en escrow via Stripe avec double systeme de commission, messagerie temps reel multi-fonctions, catalogue de services productises, gestion d'agences, notifications Push Web, facturation par abonnement, centre fiscal international, et deploiement containerise avec CI/CD automatise.

Cote technique, le backend repose sur Laravel 12 (PHP 8.2) avec Sanctum 4.3 pour l'authentification par Bearer tokens, Socialite 5.27 pour OAuth2 Google, Spatie Permissions 6.25 pour la gestion des roles, Stripe PHP SDK 20.2 pour les paiements, et Laravel Reverb pour le broadcast WebSocket. Le frontend utilise React 19 avec Vite 8, TailwindCSS 3.4, Zustand 5, Framer Motion 12, Socket.IO 4.8 et Three.js. L'intelligence artificielle est fournie localement par Ollama avec le modele Mistral 7B.

La base de donnees MySQL 8 compte 35+ tables relationnelles. L'API REST expose 100+ endpoints. Le frontend contient 80+ composants React sur 50+ pages. L'application est containerisee avec Docker (6+ services) et deployee via GitHub Actions CI/CD.

Mots-cles : Laravel 12, React 19, Stripe, Escrow, Double-entree comptable, LedgerService, WebSockets, Reverb, IA Locale, Ollama, Mistral 7B, Docker, GitHub Actions, KYC, 2FA TOTP, Marketplace Freelance, Agences, Catalogue, PFE

Table des Matieres

Remerciements

Resume

Chapitre 1 — Introduction et Contexte du Projet

Chapitre 2 — Analyse des Besoins et Specifications Fonctionnelles

Chapitre 3 — Architecture Technique Generale

Chapitre 4 — Base de Donnees — Schema Complet (35+ Tables)

Chapitre 5 — Backend Laravel 12 — Structure et Services

Chapitre 6 — Authentification, Securite, 2FA et KYC

Chapitre 7 — Marketplace : Offres, Propositions et Catalogue

Chapitre 8 — Gestion des Contrats, Jalons et Suivi du Temps

Chapitre 9 — Systeme de Paiement Escrow (LedgerService + Stripe)

Chapitre 10 — Messagerie Temps Reel (Laravel Reverb + Socket.IO)

Chapitre 11 — Intelligence Artificielle Locale (Ollama / Mistral 7B)

Chapitre 12 — Agences, Gestion des Talents et Centre Fiscal

Chapitre 13 — Administration, Finance Admin et Audit

Chapitre 14 — Frontend React 19 — Architecture et Composants

Chapitre 15 — Deploiement Docker et Pipelines CI/CD

Chapitre 16 — Tests, Qualite et Securite

Chapitre 17 — Bilan, Difficultes et Perspectives

Chapitre 1 — Introduction et Contexte du Projet

1.1 Contexte General et Motivation

Le marche mondial du travail independant (freelance) connait une croissance exponentielle. En 2025, on denombre plus de 1,57 milliard de travailleurs independants dans le monde, representant un marche global de 455 milliards de dollars, avec une croissance annuelle de 15%. Des plateformes pionnières telles qu'Upwork, Fiverr et Malt ont democratise l'acces aux talents independants. Cependant, ces plateformes presentent des limitations structurelles significatives : des commissions elevees (jusqu'a 20% chez Upwork), des interfaces vieillissantes inadaptees aux standards UX modernes, une opacite dans la gestion des paiements et une absence totale d'intelligence artificielle pour assister les utilisateurs.

C'est dans ce contexte que s'inscrit FreeNest (nom de code : Panda), un projet de fin d'etudes visant a concevoir et developper une marketplace freelance full-stack de niveau production, integrant les meilleures pratiques de l'industrie logicielle 2026.

1.2 Presentation du Projet FreeNest

FreeNest est une plateforme web SaaS (Software as a Service) concue comme une alternative moderne et complete aux plateformes etablies. Elle se distingue par :

- Une architecture technique moderne : separation stricte frontend/backend, API REST, WebSockets temps reel.
- Un systeme de paiement securise : escrow double-entree comptable, integration Stripe complete (Checkout, Connect, Webhooks).
- L'intelligence artificielle locale : integration d'Ollama/Mistral 7B sans envoi de donnees vers des services externes.
- La securite enterprise : 2FA TOTP, KYC, audit logs, rate limiting, tokens chiffres au repos.
- Le deploiement professionnel : containerisation Docker complete, pipelines CI/CD GitHub Actions.
- Un modele de commission transparent : 10% prelevee sur le freelancer, 5% sur le client, frais de retrait fixes (\$2).

1.3 Objectifs Academiques et Techniques

Ce projet vise a demontrer la maitrise des competences suivantes :

- Conception et implementation d'une API REST complete avec Laravel 12 (100+ endpoints, 21 modules).
- Developpement d'une Single Page Application (SPA) avancee avec React 19 (50+ pages, 80+ composants).
- Integration d'un systeme de paiement en escrow avec comptabilite double-entree et garanties ACID.
- Mise en place d'une infrastructure temps reel avec Laravel Reverb et Socket.IO 4.8 (11 evenements).
- Deploiement complet avec Docker, Nginx et GitHub Actions CI/CD.

- Integration d'une IA locale avec Ollama/Mistral 7B pour 5 fonctionnalites intelligentes.

1.4 Perimetre Fonctionnel

FreeNest couvre l'integralite du cycle de vie d'une mission freelance, de l'inscription jusqu'au retrait des gains, en passant par la publication d'offres, les propositions generees par IA, la gestion de contrats avec jalons, les paiements escrow via Stripe, la messagerie temps reel et l'administration complete de la plateforme.

Chapitre 2 — Analyse des Besoins et Specifications Fonctionnelles

2.1 Identification des Acteurs

Acteur	Nature	Description	Acces Principal
Freelancer	Utilisateur humain	Prestataire de services independant	Profil, Propositions, Contrats, Wallet, Catalogue, A
Client	Utilisateur humain	Entreprise ou particulier donneu	Offres, Recrutement, Escrow, Contrats, Talent Lists
Admin	Utilisateur humain	Gestionnaire de la plateforme	Dashboard, KYC, Finance, Moderation, Parametres
Google OAuth	Service externe	Fournisseur d'identite sociale	Connexion via compte Google (Socialite)
Stripe	Service externe	Passerelle de paiement internationale	Checkout, Connect Express, Webhooks, Subscripti
Ollama AI	Service local	Moteur LLM local (Mistral 7B)	Generation propositions, Matching, Chat, Analyse
Laravel Reverb	Service local	Serveur WebSocket officiel Laravel	Broadcast events, canaux prives, notifications RT

2.2 Catalogue des Besoins Fonctionnels par Module

Module Authentification et Securite

- BF-01 : Inscription par email avec validation stricte (email unique, mot de passe bcrypt)
- BF-02 : Connexion par email/mot de passe avec generation de token Sanctum
- BF-03 : Connexion sociale Google OAuth 2.0 (liaison ou creation de compte automatique)
- BF-04 : Reinitialisation du mot de passe par email (token signe, TTL 60 minutes)
- BF-05 : Activation de l'authentification a deux facteurs TOTP (Google Authenticator)
- BF-06 : Gestion des codes de secours 2FA (generation, affichage, regeneration)
- BF-07 : Onboarding guide en 5 etapes pour les nouveaux freelancers
- BF-08 : Verification d'identite KYC (upload documents, selfie, approbation admin)

Module Marketplace

- BF-09 : Publication d'offres d'emploi avec budget, type (horaire/fixe), competences
- BF-10 : Recherche full-text sur les offres (index FULLTEXT MySQL)
- BF-11 : Soumission de propositions avec jalons integres et generation par IA
- BF-12 : Catalogue de services productises (3 tiers : Basic/Standard/Premium)
- BF-13 : Commandes de services : checkout, livraison, validation, avis
- BF-14 : Recherche globale unifiee (offres + freelancers + services + agences)
- BF-15 : Autocomplete temps reel via GET /api/search/suggest

Module Contrats et Paiements

- BF-16 : Creation automatique de contrat lors de l'acceptation d'une proposition
- BF-17 : Gestion complete des jalons (funded, submitted, approved, rejected, disputed)
- BF-18 : Upload de fichiers livres avec versioning (parent_id chain)
- BF-19 : Suivi du temps horaire (start/stop timer, duration_seconds)
- BF-20 : Extensions de deadline (demande + approbation)
- BF-21 : Export PDF du contrat et du dossier de litige
- BF-22 : Depot de fonds via Stripe Checkout Session (3D Secure)
- BF-23 : Systeme escrow : fund -> release avec double-entree comptable atomique
- BF-24 : Retrait des gains via Stripe Connect Express
- BF-25 : Abonnements (plans) Stripe avec portail de facturation
- BF-26 : Factures hebdomadaires automatiques pour contrats horaires

Module Communication et IA

- BF-27 : Messagerie temps reel avec 11 evenements WebSocket (Reverb)
- BF-28 : Reactions emoji sur les messages (toggle)
- BF-29 : Accusees de lecture et de livraison en temps reel
- BF-30 : Indicateur de frappe 'UserTyping' en temps reel
- BF-31 : Notifications Push Web (VAPID + Service Worker)
- BF-32 : Generation de propositions par IA (Ollama/Mistral 7B)
- BF-33 : Matching intelligent de freelancers pour une offre
- BF-34 : Analyse et scoring du profil freelancer
- BF-35 : Chat assistant IA contextuel
- BF-36 : Recherche semantique intelligente

Module Agences, Talents et Administration

- BF-37 : Creation et gestion d'agences (invitation par token, roles owner/admin/member)
- BF-38 : Listes de talents et signets de freelancers (clients)
- BF-39 : Centre fiscal : W-9, W-8BEN, VAT — soumission et export PDF
- BF-40 : Dashboard admin : stats, gestion utilisateurs, KYC, finance, moderation

2.3 Exigences Non Fonctionnelles

Categorie	Exigence	Mesure / Critere
Performance	Reponse API < 200ms (hors IA)	Indexes DB, lazy loading, code splitting Vite
Securite	Protection multi-couches	Bearer token, 2FA, HTTPS, HMAC Webhooks, CSP
Fiabilite	Transactions ACID sur les paiements	DB::transaction(), SELECT FOR UPDATE, idempotency keys
Scalabilite	Architecture modulaire et containerisee	Docker, Redis queues, services independants
Maintenabilite	Code organise par domaine	PSR-12, 21 modules controllers, services dedie
Conformite	Traçabilité des actions sensibles	AuditLogService sur toutes les actions critiques
UX / Accessibilite	Design premium responsive	TailwindCSS, Framer Motion, dark/light mode, ARIA

Chapitre 3 — Architecture Technique Generale

3.1 Vue d'Ensemble de l'Architecture

FreeNest adopte une architecture trois-tiers strictement separee : le frontend React 19 (couche presentation), le backend Laravel 12 (couche metier et API), et la base de donnees MySQL 8 (couche donnees). Les couches communiquent exclusivement via des interfaces bien definies : REST/JSON pour les echanges synchrones, WebSocket (Laravel Reverb) pour les evenements temps reel.

3.2 Stack Technique Complet

Couche	Technologie	Version Exacte	Role dans FreeNest
API Backend	Laravel / PHP	12.x / 8.2	API REST, Business Logic, State Machines, Events
Auth Backend	Laravel Sanctum	4.3	Tokens Bearer stateless, canaux WebSocket privés
OAuth	Laravel Socialite	5.27	Connexion Google OAuth2, liaison de compte
Permissions	Spatie Permissions	6.25	Roles (freelancer/client/admin), gates Laravel
Paielements	Stripe PHP SDK	20.2	Checkout, Connect, Webhooks, Subscriptions
WebSocket	Laravel Reverb	1.10	Serveur broadcast WebSocket officiel Laravel
Images	Intervention/Image	3.11	Upload, redimensionnement, avatars
JWT	Tymon JWT-Auth	2.3	Alternative token pour certains flux
Cache/Queue	Predis / Redis	3.4	Cache sessions, queues jobs, rate limiting
UI Frontend	React / React-DOM	19.2.6	SPA, Concurrent Mode, Suspense, lazy loading
Build Tool	Vite	8.0.12	HMR, code splitting, treeshaking, assets
CSS Framework	TailwindCSS	3.4.19	Design system, dark/light mode, CSS variables
State Mgmt	Zustand	5.0.13	Stores auth, chat, theme — zero boilerplate
Animations	Framer Motion	12.39.0	Page transitions, animations fluides
Routing	React Router DOM	7.15.1	SPA routing, protected routes, lazy pages
HTTP Client	Axios	1.16.1	Interceptors Bearer token + refresh automatique
WebSocket CLI	Socket.IO Client	4.8.3	Connexion Reverb, souscription canaux privés
Echo Client	Laravel Echo	2.3.4	Abstraction Echo sur Socket.IO, auth canaux
Pusher Compat	Pusher-JS	8.5.0	Compatibilite Pusher protocol (Reverb)
Graphiques	Recharts	3.8.1	Dashboards (revenus, activite, jalons)
UI Icons	Lucide React	1.16.0	Icones coherentes, tree-shakable
UI Components	Headless UI	2.2.10	Modals, dropdowns, accessibles (ARIA)
Dates	date-fns	4.2.1	Formatage dates, calculs durees
Toasts	React Hot Toast	2.6.0	Notifications toast premium
3D	Three.js + R3F	0.170+	Globe 3D interactif landing page
Base de donnees	MySQL	8.0	35+ tables, index FULLTEXT, FK, transactions
Conteneurisation	Docker + Compose	26.x	Dev et prod containerises, 6+ services
CI/CD	GitHub Actions	latest	Tests automatisees, build, deploy sur merge
IA Moteur	Ollama	0.3+	LLM server local, API compatible OpenAI
IA Modele	Mistral 7B	7B	Modele LLM local, 7 milliards parametres

Couche	Technologie	Version Exacte	Role dans FreeNest
Proxy	Nginx	1.25	Reverse proxy, Gzip, Cache-Control, SPA

3.3 Architecture de Communication

Trois canaux de communication coexistent dans FreeNest :

- REST/JSON synchrone : 100+ endpoints HTTP sous /api/ — chaque requete inclut un Bearer token Sanctum dans le header Authorization.
- WebSocket asynchrone : Laravel Reverb diffu les evenements sur des canaux prives (conversation.{id}, user.{id}) — l'autorisation est verifiee via le middleware Sanctum avant toute souscription.
- Web Push : le Service Worker (sw.js) recoit les notifications push VAPID hors session navigateur.

3.4 Organisation du Backend en Modules

Le backend est organise en 21 modules metier, chacun dans son propre sous-dossier de Controllers/API/ :

Module	Controller(s)	Responsabilite
Auth	AuthController, PasswordResetController, TwoFactorController	Inspection, connexion, logout, 2FA, reset
AI	AIController	5 endpoints Ollama/Mistral, logs, fallback
Admin	AdminController, AdminFinanceController	Dashboard, users, KYC queue, finance, retraits
Agency	AgencyController	CRUD agences, membres, invitations token
Auth	AuthController	Inscription, connexion, OAuth, profil
Billing	SubscriptionController, WeeklyInvoiceController	Plans Stripe, abonnements, factures hebdo
Catalog	CatalogProjectController, CatalogOrderController	Services productises, commandes, livraisons
Chat	ChatController	Messages, reactions, read receipts, typing
Contracts	ContractController, MilestoneController, ContractFileController, TimeTrackingController, ContractExtensionController	Cycle de vie complet des Contrats
Freelancer	FreelancerController	Profil, onboarding, skills, portfolio, dashboard
Jobs	JobController, ProposalController	Offres emploi, propositions, statuts
KYC	IdentityVerificationController	Upload documents, file admin, approve/reject
Notifications	NotificationController	Notifications in-app, markRead, markAllRead
Payments	PaymentController, StripeController, StripeWebhookController	Retraits, sessions Stripe
Push	PushSubscriptionController	VAPID, subscribe/unsubscribe endpoints
Reviews	ReviewController	Avis post-contrat, note freelancer
Search	SearchController	Recherche globale FULLTEXT, autocomplete
Talent	SavedFreelancerController, TalentListController	Sujets freelancers, listes curees
Tax	TaxDocumentController	W-9, W-8BEN, VAT, PDF, validation admin

Chapitre 4 — Base de Donnees — Schema Complet (35+ Tables)

4.1 Principes de Conception

Le schema de la base de donnees MySQL 8 a ete concu selon les principes suivants :

- Normalisation 3NF : elimination des redondances, cles etrangeres avec ON DELETE CASCADE.
- Index FULLTEXT : ajoutes sur les champs titre, description, bio, competences pour la recherche rapide.
- Soft Deletes : utilises sur les entites sensibles (users, contracts, messages) via deleted_at.
- Precision decimale : tous les montants monetaires en DECIMAL(12,2) — jamais en FLOAT.
- Timestamps universels : created_at / updated_at sur toutes les tables, plus des timestamps metier specifiques.
- Idempotency keys : colonne idempotency_key UNIQUE sur transactions pour eviter les doubles credits.

4.2 Schema Complet par Domaine

Domaine	Tables	Champs Cles Notables
Utilisateurs	users	role, two_factor_secret (chiffre), two_factor_confirmed_at, is_
Tokens	personal_access_tokens	tokenable_id, tokenable_type, abilities, last_used_at
Profils	freelancer_profiles, client_profiles	hourly_rate, experience_level, avg_rating, total_earned, paym
Competences	categories, skills, freelancer_skills	parent_id (hierarchie), level (debutant/intermediaire/expert)
Portfolio	portfolios	images JSON, project_url, is_featured
Offres	job_postings, saved_jobs	type ENUM(hourly/fixed), budget_min/max, status ENUM, FU
Propositions	proposals	is_ai_generated, connects_used, milestones JSON, bid_amou
Contrats	contracts	type ENUM, escrow_amount DECIMAL, hourly_rate, weekly_
Activeite Contrat	contract_activities	type VARCHAR(64), data JSON, actor_id (append-only, no up
Fichiers Contrat	contract_files	parent_id (versioning chain), version INT, stored_path, mime_
Extensions	contract_extensions	old_deadline, new_deadline, reason, status ENUM(pending/a
Jalons	milestones	status ENUM, submission_notes, attachments JSON, sort_or
Temps	time_logs	started_at, ended_at (null=running), duration_seconds, screen
Finance	wallets	balance DECIMAL, pending_balance, escrow_balance — 3 b
Transactions	transactions	type ENUM(credit/debit/escrow/commission/withdrawal), direc
Retraits	withdrawals	method ENUM(bank/paypal/wise/stripe/crypto), net DECIMAL
Stripe Events	stripe_webhook_events	stripe_event_id UNIQUE (idempotence webhooks), payload J
Abonnements	subscriptions, subscription_items	stripe_id, stripe_status, plan_slug, ends_at, trial_ends_at
Factures Hebdo	weekly_invoices	week_start/end, hours DECIMAL, rate DECIMAL, total, status
Catalogue	catalog_projects, catalog_project_images, total_	is_on_order, is_available, status ENUM(pending/approved/rejec

Domaine	Tables	Champs Cles Notables
Agences	agencies, agency_members, agency_invitations	owner_id, role ENUM(owner/admin/member), token VARCHAR
Messagerie	conversations, messages, message_reactions	last_message_at, body TEXT, deleted_at (soft), reply_to_id, e
Participants	conversation_participants	last_read_at, is_muted
Avis	reviews	rating DECIMAL, breakdown JSON, contract_id
IA	ai_histories	type ENUM, input JSON, output JSON, model VARCHAR, tok
Talents	talent_lists, saved_freelancers	is_private, notes TEXT sur les membres
KYC	identity_verifications	document_type ENUM, stored paths locaux, admin_notes, rev
Fiscal	tax_documents	type ENUM(W9/W8BEN/VAT), data JSON, pdf_path, status
Audit	audit_logs	action VARCHAR, metadata JSON, ip_address, user_agent
Push	push_subscriptions	endpoint, public_key, auth_token — VAPID
Platform	platform_settings	cles de configuration : fee.freelancer_pct=0.10, fee.client_pct=
Notifications	notifications	type, notifiable_id/type (polymorphe), data JSON, read_at

4.3 Structure du Wallet a 3 Balances

Le systeme de portefeuille constitue le coeur financier de FreeNest. Chaque utilisateur dispose d'un wallet avec trois soldes distincts, garantissant la tracabilite et l'integrite financiere :

Balance	Description	Mouvements Typiques
balance	Fonds disponibles — retirables immediatement	DEBIT lors d'un depot Stripe ou liberation de jalon (-10% co
pending_balance	Fonds en attente de validation	Utilise transitoirement lors de certains transferts inter-services
escrow_balance	Fonds bloques en garantie de mission	CREDIT lors du fund-escrow client. DEBIT lors de la liberation

Le LedgerService garantit que la somme (SUM(transactions.amount * direction)) pour un wallet est toujours egale a wallet.balance — invariant verifie a la fin de chaque operation.

Chapitre 5 — Backend Laravel 12 — Structure et Services

5.1 Architecture du Backend

Le backend Laravel 12 respecte l'architecture MVC enrichie d'une couche Services qui isole la logique metier complexe des controllers. Les controllers sont reduits a leur responsabilite essentielle : valider l'entree, deleguer au service, retourner la reponse.

```
// Structure d'un controller production (ContractController) class
ContractController extends Controller { public function __construct( private
LedgerService $ledger, // money movements private NotificationService
$notifications // push + in-app ) {} public function show(Request $request,
Contract $contract): JsonResponse { $this->authorizeOrFail($request, 'view',
$contract); // ContractPolicy $contract->load(['client', 'freelancer',
'milestones', 'reviews', 'conversation']); // ... } }
```

5.2 Services Metier Dedies

Service	Responsabilites	Garanties / Particularites
LedgerService	TOUTES les operations monetaires (deposits, transferts, remboursements, etc.)	Optimisation des requetes SQL, utilisation de Redis pour le cache, mise à jour en temps réel
TotpService	Generation secrets TOTP, verification codes	Stockage sécurisé au repos via Crypt::encryptString (cle AES 256 bits)
AuditLogService	Log de toutes les actions sensibles	user_id, action, metadata JSON, ip_address, user_agent — audit trail complet
ContractActivityService	Journal d'activite des contrats (append-only)	Type-safe events : milestone.created, file.uploaded, status.changed
HourlyBillingService	Calcul et generation des factures hebdomadaires	Commande schedulee HourlyGenerateInvoicesCommand, evénement de facturation
StripeService	Sessions Checkout, Connect onboarding, idempotency key Stripe sur chaque appel API	Idempotence garantie, gestion des erreurs Stripe
SubscriptionService	Gestion lifecycle abonnements (create, swap, cancel)	Compatibilité avec Stripe Billing, Cashier-like sans package tiers
CatalogCheckoutService	Paiement d'une commande catalogue	Verifie stock tier, cree CatalogOrder, charge via LedgerService
NotificationService	Notifications in-app + broadcast + web push	Tâche asynchrone via queue, ne bloque pas la requete principale

5.3 Modele Contract et Machine d'Etat

Le modele Contract implemente une machine d'etat formelle via la constante ALLOWED_TRANSITIONS et la methode canTransitionTo(). Cela garantit que les transitions de statut sont validables avant execution et que les controllers ne peuvent jamais mettre un contrat dans un etat invalide.

```
// app/Models/Contract.php – Machine d'etat public const ALLOWED_TRANSITIONS = [
'pending' => ['active', 'cancelled'], 'active' => ['paused', 'completed',
'cancelled', 'disputed'], 'paused' => ['active', 'cancelled', 'disputed'],
'disputed' => ['active', 'completed', 'cancelled'], // admin only 'completed' =>
[], // terminal 'cancelled' => [], // terminal ]; public function
canTransitionTo(string $status): bool { return in_array($status,
self::ALLOWED_TRANSITIONS[$this->status] ?? [], true); }
```

5.4 Securite du Modele User

Le modele User est concu pour prevenir les vulnerabilites d'affectation de masse. Les champs sensibles sont volontairement exclus de fillable :

```
// app/Models/User.php – SECURITE : champs critiques hors fillable protected
$fillable = [ 'name', 'username', 'email', 'password', 'avatar', 'country',
'timezone', 'phone', 'phone_verified', 'is_online', 'last_seen_at', 'google_id',
'github_id', 'connects_balance', // NOTE : 'role', 'is_active', 'is_verified',
'email_verified_at' // sont EXCLUS intentionnellement pour prevenir l'escalade de
privileges. // Ils ne peuvent etre modifies que via des chemins codes explicitement
audites. ]; protected $hidden = ['password', 'remember_token', 'two_factor_secret',
// chiffre AES au repos 'two_factor_recovery_codes', // chiffres AES au repos ];
```

5.5 Rate Limiting par Endpoint

Groupe d'endpoints	Throttle	Justification
POST /auth/register, /auth/login	10 req/min/IP	Protection brute-force sur les credentials
POST /auth/forgot-password, /reset	5 req/min/IP	Anti-spam des emails de reinitialisation
POST /ai/*	20 req/min/user	Protection des couts LLM (Ollama calls)
GET /search, /search/suggest	60/120 req/min	Autocomplete public, couts serveur
Tous autres endpoints authentifies	Aucun (Sanctum)	Utilisateurs identifies, monitoring suffisant
POST /payments/stripe/webhook	Aucun + HMAC	Stripe verifie sa propre signature, pas de throttle

Chapitre 6 — Authentification, Securite, 2FA et KYC

6.1 Flux d'Inscription et de Connexion

FreeNest propose deux methodes d'authentification complementaires :

- Email / Mot de passe : le mot de passe est hache avec bcrypt (cost factor 12 par default). Apres validation, un token Sanctum est genere et retourne au client. Le token est stocke en localStorage cote frontend.
- Google OAuth 2.0 : via Laravel Socialite 5.27. Le controller recupere le profil Google (id, name, email, picture), cherche un utilisateur avec google_id=X ou email=Y en base, cree un nouveau compte si absent, puis genere un token Sanctum.

6.2 Processus d'Onboarding Freelancer

Apres inscription, un freelancer est redirige vers un onboarding guide en 5 etapes (page /onboarding) :

Etape	Contenu	Champs Mis a Jour
1	Categorie principale et specialites	freelancer_profiles.category, specializations
2	Niveau d'experience (entry/mid/senior)	freelancer_profiles.experience_level
3	Formation et certifications	freelancer_profiles.education, certifications JSON
4	Disponibilite et tarif horaire	freelancer_profiles.availability, hourly_rate
5	Photo de profil et langues parles	users.avatar, freelancer_profiles.languages JSON

6.3 Authentification a Deux Facteurs (2FA TOTP)

Le module 2FA utilise l'algorithme TOTP (Time-based One-Time Password, RFC 6238). L'implementation suit un flux en 3 etapes pour garantir que l'utilisateur a bien configure son application avant activation :

- POST /api/two-factor/enable : le TotpService genere un secret aleatoire (base32), le chiffre avec Crypt::encryptString (AES-256-CBC via la cle app.key Laravel), et retourne un otpauth_uri compatible Google Authenticator.
- POST /api/two-factor/confirm : l'utilisateur saisit son premier code TOTP 6 chiffres. Le TotpService le verifie avec une fenetre de 30 secondes (± 1 periode de tolerance). Si valide, two_factor_confirmed_at est defini.
- 8 codes de secours : generes lors de l'activation, chiffres au repos, presentables une seule fois. POST /api/two-factor/recovery-codes/regenerate en cree une nouvelle serie.

```
// TwoFactorController.php - enable() $secret = $this->totp->generateSecret(); //
entropie cryptographique $codes = $this->totp->generateRecoveryCodes(); // 8 codes
jetables $issuer = config('app.name') ?? 'PANDA'; $u->forceFill([
'two_factor_secret' => Crypt::encryptString($secret), 'two_factor_recovery_codes'
=> Crypt::encryptString(json_encode($codes)), 'two_factor_confirmed_at' => null,
// pas encore confirme ]->save(); $this->audit->log('user.2fa.enable_requested',
$u->id, $u);
```

6.4 Reinitialisation du Mot de Passe

Le flux de reinitialisation est implementee en deux etapes separees :

- POST /api/auth/forgot-password (throttle: 5/min) : genere un token signe (hash sha256 de donnees temporaires), envoie un email avec lien de reinitialisation valide 60 minutes.
- POST /api/auth/reset-password : valide le token, verifie son expiration, met a jour le mot de passe (bcrypt), invalide tous les tokens Sanctum de l'utilisateur (deconnexion de toutes les sessions).

6.5 Verification d'Identite KYC

Le systeme KYC (Know Your Customer) est concu pour satisfaire les exigences de conformite avant de permettre le retrait de fonds :

- L'utilisateur soumet ses documents via POST /api/verify-identity : passeport ou CNI (recto/verso) + selfie.
- Les fichiers sont stockes exclusivement sur le disque local (Storage::disk('local')) — jamais sur un disque public. Les chemins ne sont jamais exposes directement.
- L'admin accede aux fichiers via une route signee : GET /admin/kyc/file/{base64_path} qui verifie Storage::exists() avant de servir le fichier.
- Les actions KYC (approbation/rejet) sont loguees dans audit_logs via AuditLogService.
- L'approbation KYC met a jour is_verified=true sur l'utilisateur, debloquant les retraits.

6.6 Securite Globale

Mesure	Implementation	Protection Contre
HMAC Signature Webhooks	Stripe::constructEvent(payload, sig, secret)	Faux webhooks injectes
Tokens chiffres au repos	Crypt::encryptString(AES-256-CBC)	Vol de secrets 2FA en base
Mass Assignment Guard	fillable whitelist sur User (role hors liste)	Privilege escalation via \$request->all()
Audit Trail	AuditLogService sur actions sensibles	Non-repudiation, forensique
Input Validation	Form Requests Laravel sur 100% des endpoints	Injection, XSS, donnees invalides
ContractPolicy (Gates)	authorizeOrFail() dans chaque controller	Acces non autorise aux ressources
Middleware admin	Double verification controller + middleware	Acces admin depuis endpoint non admin

Chapitre 7 — Marketplace : Offres, Propositions et Catalogue

7.1 Gestion des Offres d'Emploi

Les offres d'emploi (job_postings) constituent le coeur de la marketplace. Chaque offre est publiee par un client avec les attributs suivants : titre, description detaillee, categorie, type (hourly/fixed), fourchette budgetaire (budget_min/budget_max), competences requises, et statut.

Le cycle de vie d'une offre suit une machine d'etat :

Statut	Acces Marketplace	Transition Vers
draft	Non visible	open (publication client)

Statut	Acces Marketplace	Transition Vers
open	Visible, proposals OK	in_progress (proposition acceptee), cancelled
in_progress	Non visible (en cours)	completed (jalons terminees), cancelled
completed	Archive	Etat terminal
cancelled	Archive	Etat terminal

7.2 Recherche et Filtrage

La recherche utilise des index FULLTEXT MySQL sur titre, description et competences pour des performances optimales :

```
// SearchController.php – Recherche FULLTEXT $results = JobPosting::whereRaw(
'MATCH(title, description) AGAINST(? IN BOOLEAN MODE)', [$request->q . '*']
)->where('status', 'open') ->with('category') ->paginate(20);
```

L'autocomplete (GET /api/search/suggest, throttle 120/min) retourne des suggestions instantanees sur offres, freelancers, services et agences, agregees en une seule reponse JSON.

7.3 Systeme de Propositions

Les freelancers soumettent des propositions avec : lettre de motivation, montant propose (bid_amount), decoupe en jalons (milestones JSON) et un flag is_ai_generated pour la transparence. L'acceptation d'une proposition par le client declenche automatiquement :

- Mise a statut 'accepted' de la proposition selectionnee.
- Rejet automatique de toutes les autres propositions pour cette offre.
- Creation d'un contrat (INSERT contracts) avec les donnees de la proposition.
- Creation des jalons en base depuis le JSON milestones de la proposition.
- Creation d'une conversation liee au contrat.
- Envoi de notifications aux deux parties.

Toutes ces operations sont enveloppees dans une seule DB::transaction() — atomicite garantie.

7.4 Catalogue de Services (Style Fiverr)

En complement des offres, FreeNest propose un catalogue de services productises ou les freelancers peuvent vendre des prestations standardisees directement :

Entite	Description	Champs Cles
CatalogProject	Service cree par un freelancer	tiers JSON (Basic/Standard/Premium avec prix et delai), statut
CatalogOrder	Commande d'un tier par un client	tier ENUM, price DECIMAL, status ENUM(pending/in_progress)
CatalogReview	Avis post-commande	rating (1-5), comment, commande liee
Moderation	Validation admin avant publication	POST /admin/catalog/{id}/approve ou /reject

Chapitre 8 — Gestion des Contrats, Jalons et Suivi du Temps

8.1 Architecture du Systeme de Contrats

Le systeme de contrats est le module le plus complexe de FreeNest. Il implique 7 controllers dedies, 5 modeles interconnectes, un journal d'activite append-only, et une machine d'etat formelle. La logique monetaire est systematiquement deleguee au LedgerService pour garantir l'atomicite des transactions financieres.

8.2 Cycle de Vie d'un Contrat

Statut	Description	Acteur	Prochains Etats Possibles
pending	En attente de confirmation	Systeme	active, cancelled
active	Mission en cours	Systeme	paused, completed, cancelled, disputed
paused	Travail temporairement suspendu	Parties	active, cancelled, disputed
completed	Tous jalons approuves, mission systeme	Systeme	Aucun (terminal)
disputed	Litige ouvert par une partie	Partie	active, completed, cancelled (admin)
cancelled	Annulation mutuellement convenue	Parties/Admin	Aucun (terminal)

8.3 Fonctionnalites Avancees des Contrats

Fonctionnalite	Endpoint	Details Techniques
Fichiers livres	POST /contracts/{id}/files	Versioning via parent_id chain. Colonnes : original_name, sto
Suivi du temps	POST /contracts/{id}/time/start stop	time_logs : started_at, ended_at (null=timer actif), duration_se
Extensions deadline	POST /contracts/{id}/extensions	contract_extensions : old_deadline, new_deadline, reason, sta
Analytics	GET /contracts/{id}/analytics	Stats temps passe (sum duration_seconds), jalons par statut,
Journal d'activite	GET /contracts/{id}/activity	contract_activities : append-only. Types : milestone.created, fi
Export PDF	GET /contracts/{id}/pdf	PDF signe du contrat (titre, parties, montant, jalons, conditions
PDF Litige	GET /contracts/{id}/dispute-pdf	PDF du dossier de litige (motif, historique, preuves) pour arbit
Archive	POST /contracts/{id}/archive	Positionne archived_at (timestamp). Exclut des listes par defa

8.4 Workflow Detaille des Jalons

Chaque jalon suit son propre cycle de vie independant. Le LedgerService gere les mouvements financiers associes :

Statut Jalon	Acteur Responsable	Operation LedgerService	Description
pending	Systeme (creation)	Aucune	Cree avec le contrat depuis le JSON milestones de
funded	Client	fundEscrow(client, contract, amount)	balance -= amount, escrow_balance += amount, co
submitted	Freelancer	Aucune (document + notes)	Freelancer soumet via POST /milestones/{id}/subm
approved	Client	releaseMilestone(client, milestone)	escrow -= amount, freelancer.balance += 90%, plat
rejected	Client	Aucune (retour a funded)	Client demande des corrections via POST /mileston
disputed	Admin	Selon resolution	resolved pour freelancer: approve, resolved pour cl

8.5 Suivi du Temps et Facturation Horaire

Pour les contrats de type 'hourly', FreeNest implemente un systeme de time tracking :

- POST /contracts/{id}/time/start : cree un TimeLog avec started_at=now(), ended_at=NULL (timer actif).
- POST /contracts/{id}/time/stop : calcule duration_seconds = ended_at - started_at, ferme le TimeLog.
- GET /contracts/{id}/time/weekly : retourne le resume hebdomadaire (heures, montant a facturer).
- Commande Laravel schedulee (HourlyGenerateInvoicesCommand) : genere automatiquement les WeeklyInvoice chaque lundi matin, diffuse les evenements WeeklyInvoiceGenerated et WeeklyInvoicePaid via Reverb.

Chapitre 9 — Systeme de Paiement Escrow (LedgerService + Stripe)

9.1 Les 6 Garanties du LedgerService

Le LedgerService constitue la piece maitresse du systeme financier de FreeNest. Il est concu selon 6 garanties immuables, documentees dans son en-tete de classe :

- Atomicite : chaque operation monetaire s'execute dans une DB::transaction(). En cas d'erreur, tout est annule.
- Verrouillage : les lignes wallet sont verrouillees avec SELECT ... FOR UPDATE avant mutation, eliminant les race conditions en environnement concurrent.
- Double-entree : chaque mouvement de fonds genere au minimum deux entrees en transaction (debit + credit). La plateforme a son propre compte systeme (is_platform=true) qui recoit les commissions.
- Audit immutable : chaque transaction stocke balance_after DECIMAL — permettant de reconcilier l'historique et de detecter toute alteration.
- Idempotence : la colonne idempotency_key UNIQUE sur transactions garantit que l'envoi multiple d'une meme requete (retry client) ne credite qu'une seule fois.
- Invariant verifie : a la fin de chaque operation, le service verifie SUM(transactions) == wallet.balance.

9.2 Structure des Commissions

Le modele de commission de FreeNest est configure dans la table platform_settings et est entierement administrable :

Cle Setting	Valeur par Defaut	Description
fee.freelancer_pct	0.10 (10%)	Commission prelevee sur le paiement reçu par le freelancer
fee.client_pct	0.05 (5%)	Frais de service factures au client en plus du montant contractuel
fee.contract_init	\$0.99	Frais d'initiation de contrat factures au client a la creation
fee.withdrawal_flat	\$2.00	Frais fixes deduits de chaque demande de retrait
fee.deposit_pct	0.029 (2.9%)	Frais de traitement Stripe sur les depots par carte
fee.deposit_flat	\$0.30	Frais fixes Stripe sur chaque depot

Cle Setting	Valeur par Defaut	Description
withdrawal.min	\$20.00	Montant minimum pour une demande de retrait

9.3 Flux Complet de Paiement Escrow

Illustration du flux de bout en bout pour un jalon de \$300 :

```
// Phase 1 : Depot client via Stripe Checkout
StripeController::createDepositSession() ->
Stripe::checkout.session.create({amount: 30000, currency: 'usd'}) -> Redirect
client vers checkout.stripe.com/... -> Stripe webhook: checkout.session.completed
-> LedgerService::deposit(client, 300.00) wallet.balance = 300.00 |
Transaction(type:credit, amount:300, balance_after:300) // Phase 2 : Financement
escrow par le client LedgerService::fundEscrow(client, contract, 300.00) -> SELECT
wallets WHERE user_id=client FOR UPDATE -> wallet.balance = 0.00 |
wallet.escrow_balance = 300.00 -> contract.escrow_amount = 300.00 ->
Transaction(type:escrow, direction:out, amount:300, balance_after:0) // Phase 4 :
Liberation par le client (apres approbation du jalon)
LedgerService::releaseMilestone(client, milestone) -> commission = 300 * 0.10 =
30.00 (fee.freelancer_pct) -> net_freelancer = 300 - 30 = 270.00 ->
wallet(client).escrow_balance = 0.00 -> wallet(freelancer).balance = 270.00 ->
wallet(platform).balance += 30.00 -> Transaction(type:commission, amount:30,
user:platform) -> Transaction(type:credit, amount:270, user:freelancer,
balance_after:270)
```

9.4 Stripe Connect et Retraits

Les freelancers peuvent retirer leurs gains vers leur compte bancaire via Stripe Connect Express :

- POST /payments/stripe/connect/onboard : cree une session Stripe Connect Express onboarding. L'URL est retournée pour redirection.
- GET /payments/stripe/connect/status : verifie charges_enabled et payouts_enabled sur le compte Stripe Connect.
- POST /payments/withdrawals : cree une demande de retrait (Withdrawal). Si withdrawal.requires_approval=1, soumis a l'admin. Sinon, declenche directement le transfert Stripe.
- L'admin valide via POST /admin/finance/withdrawals/{id}/approve qui declenche un Stripe Transfer vers le compte Connect du freelancer.

9.5 Gestion des Webhooks Stripe

Le StripeWebhookController (POST /payments/stripe/webhook) est le seul endpoint sans authentification Sanctum. Il utilise la verification de signature Stripe (HMAC-SHA256) pour garantir l'authenticite des evenements :

Evenement Stripe	Action dans FreeNest
checkout.session.completed	LedgerService::deposit() — credite le wallet du client
account.updated	Met a jour stripe_connect_status de l'utilisateur freelancer
invoice.paid	Confirme le paiement d'un abonnement, active subscription
invoice.payment_failed	Marque subscription en statut past_due, notifie l'utilisateur
customer.subscription.deleted	Desactive l'abonnement (ends_at = now())

Chaque evenement Stripe est stocke dans stripe_webhook_events avec stripe_event_id UNIQUE — garantissant l'idempotence meme si Stripe livre le meme evenement plusieurs fois (retry policy Stripe).

9.6 Abonnements et Plans

FreeNest propose un systeme d'abonnement pour des plans premium (plus de connects, mise en avant dans la recherche, acces au catalogue IA complet) :

- GET /api/plans : catalogue public des plans (retourne des fixtures ou des produits Stripe en live).
- POST /api/billing/checkout : cree une session Stripe Checkout pour souscrire a un plan.
- POST /api/billing/swap : change de plan (upgrade immediatement, downgrade en fin de periode).
- GET /api/billing/portal : URL du portail Stripe pour gerer la carte, les factures, l'annulation.
- Middleware EnsurePlan : bloque l'acces a certains endpoints si l'utilisateur n'a pas le plan requis.

Chapitre 10 — Messagerie Temps Reel (Laravel Reverb + Socket.IO)

10.1 Architecture WebSocket

La messagerie temps reel de FreeNest repose sur Laravel Reverb 1.10, le serveur WebSocket officiel de Laravel introduit en 2024. Reverb implemente le protocole Pusher Channels, permettant d'utiliser les librairies Laravel Echo et Pusher-JS cote client.

Architecture de la connexion :

- Backend : Laravel diffuse des evenements via Event::dispatch() sur des canaux prives (PrivateChannel).
- Reverb : recoit l'evenement, verifie l'autorisation du canal (via routes/channels.php + Sanctum), puis diffu aux clients abones.
- Frontend : laravel-echo 2.3.4 + pusher-js 8.5.0 + socket.io-client 4.8.3 s'abonnent aux canaux prives.

10.2 Catalogue Complet des Evenements WebSocket

Evenement	Canal	Payload Diffuse	Declencheur
message.sent	conversation.{id}	id, sender_id, body, type, attachments	POST /chat/{id}/send
message.edited	conversation.{id}	id, body, edited_at	PUT /chat/messages/{id}
message.deleted	conversation.{id}	id, deleted_at	DELETE /chat/messages/{id}
message.read	conversation.{id}	reader_id, conversation_id, read_at	POST /chat/{id}/read
message.delivered	conversation.{id}	message_id, delivered_at	POST /chat/messages/{id}/delivered
message.reaction_toggled	conversation.{id}	message_id, emoji, user_id, action	POST /chat/messages/{id}/reactions
user.typing	conversation.{id}	user_id, name, is_typing (bool)	POST /chat/{id}/typing
conversation.updated	conversation.{id}	last_message, unread_count, updated_at	PUT /chat/{id}/update

Evenement	Canal	Payload Diffuse	Declencheur
notification.created	user.{id}	id, type, data JSON, created_at	NotificationService::send()
weekly_invoice.generated	user.{id}	invoice_id, week_start, total	HourlyGenerateInvoicesCommand
weekly_invoice.paid	user.{id}	invoice_id, paid_at	Payment confirmation

10.3 Securite des Canaux WebSocket

Tous les canaux utilises sont des PrivateChannel — jamais des canaux publics. L'autorisation est verifiee dans routes/channels.php :

```
// routes/channels.php Broadcast::channel('conversation.{id}', function (User
$user, int $id) { // Seuls les participants de la conversation peuvent s'abonner
return $user->conversations()->where('id', $id)->exists(); });
Broadcast::channel('user.{id}', function (User $user, int $id) { // Un utilisateur
ne peut s'abonner qu'a son propre canal return (int) $user->id === $id; });
```

Avant de s'abonner, le client frontend envoie une requete POST /broadcasting/auth avec son Bearer token Sanctum. Reverb verifie l'autorisation et retourne un token de souscription signe.

10.4 Payload Optimise des Evenements

Les payloads des evenements sont volontairement minimalistes (exemple: MessageSent) :

```
// Events/MessageSent.php – broadcastWith() public function broadcastWith(): array
{ // Payload slim – jamais le modele Eloquent complet // Cela evite de leaked des
champs sensibles et de casser // les clients Echo si le schema change. return [
'id' => $m->id, 'conversation_id' => $m->conversation_id, 'sender_id' =>
$m->sender_id, 'body' => $m->body, 'type' => $m->type, 'is_read' => (bool)
$m->is_read, 'created_at' => optional($m->created_at)->toIso8601String(), ]; }
```

10.5 Notifications Push Web (VAPID)

En complement des notifications temps reel in-app, FreeNest implemente les Web Push Notifications via le standard VAPID (Voluntary Application Server Identification) :

- sw.js (Service Worker) : enregistre cote navigateur, ecoute les evenements push et les affiche meme quand l'onglet est ferme.
- POST /api/push/subscribe : enregistre l'endpoint push du navigateur (endpoint, public_key, auth_token) dans push_subscriptions.
- GET /api/push/vapid-public-key : retourne la cle VAPID publique pour la souscription navigateur.
- browserNotify.js : utilise l'API Notification du navigateur avec icone et son (sound.js) pour les notifications foreground.

Chapitre 11 — Intelligence Artificielle Locale (Ollama / Mistral 7B)

11.1 Choix de l'IA Locale

Contrairement aux SaaS d'IA (OpenAI GPT-4, Anthropic Claude) qui necessitent l'envoi des donnees utilisateurs vers des serveurs externes et engendrent des couts variables,

FreeNest integre Ollama avec le modele Mistral 7B en local. Ce choix offre : confidentialite totale des donnees utilisateurs, couts d'infrastructure previsibles, disponibilite hors internet et possibilite de personnalisation du modele.

Ollama expose une API REST compatible (POST /api/generate) que le AIController interroge via le client HTTP Laravel (Http::timeout(30)->post()).

11.2 Les 5 Fonctionnalites IA

Endpoint	Fonctionnalite	Entrees	Prompt Envoye a Ollama
POST /ai/generate-proposal	Generation de proposition	job_id, freelancer_summary optional	Construit a partir du profil freelancer + details offre
POST /ai/match-freelancers	Matching de freelancers	job_id	N'appelle pas Ollama — utilise un scoring algorithm
POST /ai/analyze-profile	Analyse de profil	Profil utilisateur actuel	Demande JSON structure avec : score (0-100), stre
POST /ai/chat	Chat assistant IA	message (max 2000 chars)	System prompt PANDA AI assistant. Repond sur la
POST /ai/smart-search	Recherche intelligente	query (string libre)	Convertit une requete langage naturel en paramet

11.3 Gestion de la Disponibilite et Fallback

Le AIController implemente un mecanisme de fallback robuste en cas d'indisponibilite d'Ollama (service arrete, timeout, erreur reseau) :

```
// AIController.php – callAI() avec fallback
private function callAI(string $prompt, string $system = ''): string {
    try {
        $response = Http::timeout(30)->post(
            "{$this->ollamaUrl}/api/generate",
            [
                'model' => $this->defaultModel,
                // 'mistral'
                'prompt' => ($system ? "System: $system " : '') . "User: $prompt",
                'stream' => false,
            ]
        );
        if ($response->successful()) {
            return $response->json('response', 'AI service temporarily unavailable. Please try again.');
```

11.4 Traçabilité et Audit des Appels IA

Chaque appel IA (hors matching algorithmique) est logue dans ai_histories avec :

- user_id : identifiant de l'utilisateur demandeur.
- type : enum (proposal / chat / analyze / smart_search).
- input : JSON des parametres d'entree (job_id, message, etc.).
- output : JSON de la reponse generee.
- model : modele utilise ('mistral').
- tokens_used : estimation (len(reponse) / 4) — permet un audit de consommation.

Le rate limiting (20 req/min via throttle:20,1) protege contre les abus et les couts excessifs.

Chapitre 12 — Agences, Gestion des Talents et Centre Fiscal

12.1 Systeme d'Agences

FreeNest permet a des groupes de freelancers de se regrouper sous une agence avec une marque commune, une page publique et une gestion des membres par roles. Ce systeme

repond a la demande croissante d'equipes freelance collaboratives.

Fonctionnalite	Endpoint	Details
Creation	POST /agencies	Le createur devient owner. Stocke : name, slug, description, lo
Invitation	POST /agencies/{id}/invitations	Envoie email avec token unique (agency_invitations.token VA
Acceptation	POST /agencies/invitations/{token}/accept	Verifie token, cree agency_members(role=member), supprime
Refus	POST /agencies/invitations/{token}/decline	Supprime l'invitation sans rejoindre l'agence.
Gestion membres	GET /agencies/{id}/members	Liste avec roles. DELETE /agencies/{id}/members/{uid} pour r
Transfert propriete	POST /agencies/{id}/transfer-ownership	Change owner_id. Nouveau propriétaire devient owner, ancien
Repertoire public	GET /agencies, GET /agencies/{id}	Endpoints publics — acces sans authentication.

12.2 Gestion des Talents (Clients)

Les clients disposent de deux outils complementaires pour organiser leur vivier de freelancers :

- Saved Freelancers : signet rapide (un clic) via POST /saved-freelancers. Accessible via GET /saved-freelancers avec verif d'existence via GET /saved-freelancers/check/{freelancer_id}.
- Talent Lists : listes nommees et curees (ex: 'Designers UI React', 'Developpeurs API Senior'). CRUD complet + ajout/retrait de membres individuels.

12.3 Centre Fiscal International

Le TaxCenter permet aux utilisateurs de gerer leurs obligations fiscales selon leur juridiction :

Type	Usage	Champs du Formulaire
W-9	Travailleurs independants residant aux USA	Nom legal, TIN (SSN/EIN), adresse, certification
W-8BEN	Etrangers percevant des revenus de source americaine	Pays de residence, numero d'identification fiscal etranger
VAT	Entreprises europeennes soumises a la TVA	Numero de TVA intracommunautaire, pays, montant

Les documents soumis peuvent etre exportes en PDF via GET /tax-documents/{id}/pdf. L'admin valide ou rejette via POST /admin/tax-documents/{id}/approve|reject avec motif.

Chapitre 13 — Administration, Finance Admin et Audit

13.1 Double Protection de l'Espace Admin

L'espace admin est protege par deux couches independantes de securite pour garantir la defense en profondeur :

- Middleware 'admin' : verifie role === 'admin' sur le User authentifie avant chaque requete. Retourne HTTP 403 si la condition n'est pas satisfaite.
- Verification dans le controller : chaque methode admin verifie independamment le role. Un contournement du middleware n'est donc pas suffisant.

13.2 Dashboard Administrateur

- Statistiques globales : nombre total d'utilisateurs (par role), offres actives, contrats en cours, GMV (Gross Merchandise Value) du mois, commissions perçues.
- Gestion des utilisateurs : liste paginée, filtres par role/statut, actions ban (is_active=false) et verify (is_verified=true) avec audit log.
- Analytics avancées : croissance des inscriptions, volume de transactions, taux de completion des contrats.

13.3 Finance Admin

Endpoint Admin Finance	Methode	Action
/admin/finance/dashboard	GET	Vue d'ensemble : balance plateforme, commissions jour/mois
/admin/finance/withdrawals	GET	File des demandes de retrait (status=pending) avec details : u
/admin/finance/withdrawals/{id}/approve	POST	Approuve retrait -> LedgerService -> Stripe Transfer -> status
/admin/finance/withdrawals/{id}/reject	POST	Rejette avec motif -> rembourse escrow_balance -> notifie fre
/admin/finance/settings	GET	Lecture de tous les platform_settings (cles de commission, lim
/admin/finance/settings	PUT	Mise a jour d'un ou plusieurs platform_settings (ex: modifier fe

13.4 AuditLogService et Conformite

Le AuditLogService est injecte dans tous les controllers qui effectuent des operations sensibles. Chaque entree dans audit_logs contient :

```
// AuditLogService::log() – signature public function log(string $action, int $userId, mixed $context = null): void { AuditLog::create([ 'user_id' => $userId, 'action' => $action, // ex: 'user.2fa.enable_requested', 'kyc.approved' 'metadata' => json_encode($context), // objet ou array contextuel 'ip_address' => request()->ip(), 'user_agent' => request()->userAgent(), ]); }
```

Actions auditees : connexion/deconnexion, activation 2FA, soumission KYC, approbation/rejet KYC, approbation/rejet retrait, modification des parametres plateforme, ban/unban utilisateur.

Chapitre 14 — Frontend React 19 — Architecture et Composants

14.1 Structure du Projet Frontend

```
frontend-react/ src/ pages/ # 50+ pages organisees par domaine fonctionnel Auth/ # Login, Register, Onboarding, ForgotPassword, ResetPassword, GoogleCallback Dashboard/ # FreelancerDashboard, ClientDashboard Admin/ # AdminDashboard, AdminFinance Jobs/ # JobsMarketplace, JobDetail, PostJob, CategoryJobs, MyJobs, MyProposals Freelancer/ # FreelancerProfile, FreelancerSettings, FreelancerMarketplace Contracts/ # ContractsList, ContractDetails Milestones/ # MilestoneDetails Chat/ # Messages (temps reel) Payments/ # Payments, Withdraw, PaymentSuccess, PaymentCancel, StripeConnectReturn Billing/ # Billing (abonnements et factures) Settings/ # TwoFactorSettings, IdentityVerification, TaxCenter Talent/ # SavedFreelancers, TalentLists, TalentListDetails Search/ # Search, GlobalSearch Agencies/ # Agencies AI/ # AIAssistant Reports/ # Reports Landing/ # Landing (globe 3D, Three.js) Pricing/ # Pricing Resources/ # Blog, HowItWorks, Reviews, Updates,
```

```

SuccessStories, Research GetOutcomes/ # GetOutcomes, BuildWebsite, ScalePaidAds,
HandleSupport FindTalent/ # FindTalent components/ # 80+ composants reutilisables
layout/ # AppLayout, AuthLayout, GlobalNavbar, Sidebar contracts/ # ActivityTab,
AnalyticsTab, ChatTab, ContractActions, DisputeModal, ExtensionsTab, FilesTab,
MilestonesTab, RatingModal, TimeTrackingTab, ContractTimeline, ContractStatusBadge
milestones/ # CreateMilestoneModal, MilestoneCard, MilestoneList,
MilestoneStatusBadge, MilestoneTimeline, RejectWorkModal, SubmitWorkModal ui/ #
NavSearch, ... composants generiques store/ # Zustand stores authStore.js # token,
user, fetchMe(), login(), logout() chatStore.js # conversations, messages, unread
counts themeStore.js # theme (dark/light), applyTheme() api/ axios.js # Instance
Axios avec interceptor Bearer token auto index.js # 200+ fonctions API (une par
endpoint) lib/ echo.js # Configuration Laravel Echo + Reverb browserNotify.js # API
Notification navigateur + son sound.js # Sons de notification (new message, etc.)
webPush.js # Service Worker VAPID registration utils/ footerLinks.js # Liens de
navigation footer

```

14.2 Gestion d'Etat avec Zustand

Zustand 5.0 est utilise pour l'etat global de l'application avec trois stores independants :

Store	Etat Gere	Actions Principales
authStore	token (Bearer), user (profil complet), isLoading	fetchMe() au demarrage si token present, login(token), logout
chatStore	conversations [], messages {}, unreadCounts	Mises a jour temps reel via Echo events (message.sent, mess
themeStore	theme ('dark' 'light'), systemPreference	applyTheme() applique les CSS variables sur :root — Tailwind

14.3 Routing et Protection des Routes

React Router DOM 7.15.1 gere le routing SPA avec trois types de routes :

- Routes publiques : accessible sans authentication (/ , /jobs, /freelancers, /pricing, etc.).
- GuestRoute : redirige vers /dashboard si token present (evite les boucles de connexion).
- ProtectedRoute(roles) : verifie token + role autorise — redirige vers /login si absent, /dashboard si mauvais role.

Toutes les pages sont lazy-loaded (import()) dynamique) avec Suspense et un PageLoader personnalise, garantissant que le bundle initial est minimal.

14.4 Configuration Laravel Echo pour Reverb

```

// src/lib/echo.js - Configuration Echo + Reverb import Echo from 'laravel-echo';
import Pusher from 'pusher-js'; window.Pusher = Pusher; const echo = new Echo({
broadcaster: 'reverb', // protocole Pusher/Reverb key:
import.meta.env.VITE_REVERB_APP_KEY, wsHost: import.meta.env.VITE_REVERB_HOST,
wsPort: import.meta.env.VITE_REVERB_PORT, wssPort:
import.meta.env.VITE_REVERB_PORT, forceTLS: false, enabledTransports: ['ws',
'wss'], authEndpoint: '/broadcasting/auth', auth: { headers: { Authorization:
'Bearer ' + localStorage.getItem('token'), }, }, }); // Usage dans un composant
React : // echo.private(`conversation.${convId}`) // .listen('message.sent',
(data) => { updateMessages(data); });

```

Chapitre 15 — Deploiement Docker et Pipelines CI/CD

15.1 Containerisation Docker

L'ensemble de l'application FreeNest est containerisee avec Docker, garantissant la reproductibilite des environnements et l'absence de dependances systeme non declarees. Le docker-compose.yml orchestre 6 services :

Service Docker	Image / Dockerfile	Ports	Role
backend	Dockerfile multi-stage (PHP 8.2 + Nginx)	8000	API Laravel 12, queues, scheduler
frontend	Dockerfile Nginx (Vite build static)	5173	SPA React 19 — nginx.conf SPA
mysql	mysql:8.0	3306	Base de donnees principale
redis	redis:7-alpine	6379	Cache sessions, queues Laravel, rate limiting
reverb	Via backend (php artisan reverb)	8080	Serveur WebSocket Laravel Reverb
ollama	ollama/ollama	11434	LLM local Mistral 7B

15.2 Configuration Nginx Production

Le nginx.conf du frontend est optimise pour une SPA React avec Service Worker :

```
# nginx.conf — Points cles server { listen 80; root /usr/share/nginx/html; #
Compression Gzip des assets gzip on; gzip_types text/plain application/javascript
text/css application/json; # Cache long terme pour assets hashe (Vite les hashe
automatiquement) location ~* \.(js|css|woff2|png|jpg|svg)$ { expires 1y; add_header
Cache-Control "public, immutable"; } # index.html jamais cache (reference les
assets hashe) location = /index.html { add_header Cache-Control "no-cache,
no-store, must-revalidate"; } # Service Worker doit etre servi sans cache location
= /sw.js { add_header Cache-Control "no-cache"; } # SPA routing — toutes les routes
-> index.html location / { try_files $uri $uri/ /index.html; } }
```

15.3 Pipelines CI/CD GitHub Actions

Deux workflows YAML automatisent le cycle de livraison :

backend.yml — declenche sur push/PR vers main :

- Checkout du code (actions/checkout@v4)
- Setup PHP 8.2 avec extensions requises (pdo_mysql, redis, gd)
- Installation des dependances (composer install --no-dev --optimize-autoloader)
- PHP CS Fixer : verification de conformite PSR-12
- PHPStan : analyse statique niveau 6 (detection d'erreurs de types)
- Tests PHPUnit : php artisan test --coverage (tests unitaires et integration)
- Build image Docker et push vers le registry

frontend.yml — declenche sur push/PR vers main :

- Checkout du code
- Setup Node.js 20 LTS
- Installation des dependances (npm ci)
- ESLint : lint du code JavaScript/JSX
- Vite build : npm run build — echec si erreur TS/JSX
- Vitest : tests unitaires des composants React

Note : Un echec a n'importe quelle etape des deux pipelines bloque automatiquement le merge de la Pull Request.

Chapitre 16 — Tests, Qualite et Securite

16.1 Tests Backend (PHPUnit)

- Tests unitaires des services : LedgerService (scenarios deposit, fundEscrow, releaseMilestone, cas erreur insolvabilite), TotpService (generation/verification codes TOTP).
- Tests d'integration des endpoints : HTTP tests Laravel — simulation requetes authentifiees et non-authentifiees.
- Test Stripe Webhook : verification que les requetes sans signature HMAC valide retournent HTTP 400.
- Test machine d'etat contrat : verification que les transitions invalides retournent HTTP 422.

16.2 Tests Frontend (Vitest)

- Tests composants : React Testing Library — rendu, interactions utilisateur (clic, saisie).
- Tests stores Zustand : authStore login/logout, etat initial.
- Tests fonctions API : mock Axios, verification des parametres envoyes.

16.3 Outils de Qualite du Code

Outil	Cible	Configuration	Objectif
PHP CS Fixer	Backend PHP	PSR-12 strict	Formatage uniforme, style coherent
PHPStan	Backend PHP	Niveau 6 (sur 9)	Detection types manquants, nulls non geres
Laravel Pint	Backend PHP	Preset Laravel	Auto-formatage integre Laravel
ESLint 10	Frontend React	Plugin react-hooks, react-refresh	Regles hooks, imports
Prettier	Frontend	Via eslint-plugin	Formatage automatique coherent
Vite TypeScript	Frontend	@types/react 19.2.14	Type checking JSX/TSX

16.4 Vulnerabilites OWASP Couvertes

Vulnerabilite OWASP	Mesure de Protection Implementee
A01 Broken Access Control	ContractPolicy, middleware admin, canTransitionTo(), PrivateChannel authorization
A02 Cryptographic Failures	bcrypt mots de passe, AES-256 secrets 2FA, HTTPS obligatoire, HMAC webhooks
A03 Injection	Eloquent ORM (requetes preparees), FormRequests Laravel (validation stricte)
A05 Security Misconfiguration	is_platform user non-logable, role hors fillable, .env hors vcs
A07 Auth Failures	Throttle login, 2FA TOTP, tokens Sanctum, expiration sessions
A08 Data Integrity Failures	Idempotency keys, transactions ACID, HMAC Stripe, balance_after audit
A09 Logging Failures	AuditLogService sur toutes les actions sensibles, logs Laravel structurés

Chapitre 17 — Bilan, Difficultes et Perspectives

17.1 Bilan Quantitatif du Projet

Metrique	Valeur	Commentaire
Tables MySQL	35+	Schema complet, normalise 3NF, FK constraints
Migrations Laravel	30+	Historique complet du schema, rollback possible
Endpoints REST API	100+	Documentes, valides, throttled
Controllers	21 modules	Organises par domaine, thin controllers
Services metier	9	LedgerService, TotpService, AuditLogService...
Modeles Eloquent	40+	Relations, casts, scopes, soft deletes
Evenements WebSocket	11	PrivateChannel, broadcastWith() slim
Pages React	50+	Lazy-loaded, protected routes, role-based
Composants React	80+	Reutilisables, organisees par domaine
Fonctionnalites IA	5	Ollama/Mistral 7B, fallback, rate limit
Services Docker	6+	Backend, Frontend, MySQL, Redis, Reverb, Ollama
Pipelines CI/CD	2	backend.yml + frontend.yml GitHub Actions
Lignes de code (total)	~25 000+	Backend PHP + Frontend JSX + migrations + configs
Packages backend (prod)	8	Laravel, Sanctum, Socialite, Stripe, Reverb...
Packages frontend (prod)	14	React, Vite, Zustand, Framer Motion, Echo...

17.2 Difficultes Rencontrees et Solutions

Difficulte	Impact	Solution Implementee
Race conditions sur les paiements escrow	Critique	SELECT ... FOR UPDATE sur le wallet + DB::transaction() da
Idempotence des webhooks Stripe	Critique	Colonne stripe_event_id UNIQUE dans stripe_webhook_even
Securite 2FA : secrets au repos	Elevee	Crypt::encryptString() (AES-256-CBC, cle app.key). Les secre
Autorisation canaux WebSocket	Elevee	routes/channels.php + PrivateChannel — verification member
FULLTEXT search performances	Moyenne	Index FULLTEXT MySQL via migration 2026_06_08_000003.
Transactions atomiques multi-etapes	Elevee	Toutes les acceptations de propositions (6 operations) dans u
Fichiers KYC sensibles	Critique	Storage::disk('local') (non-public), acces via route signee avec

17.3 Competences Acquises

- Architecture logicielle : separation des responsabilites, services, repositories, machines d'etat.
- API REST production : validation, autorisation par policy, pagination, rate limiting, versionning.
- Comptabilite numerique : systeme double-entree, invariants financiers, idempotence, ACID.
- Securite applicative : OWASP Top 10, 2FA TOTP, KYC, audit trail, chiffrement au repos.
- Temps reel : WebSockets (Reverb), canaux prives, optimisation des payloads evenements.

- Infrastructure : Docker multi-service, Nginx SPA, GitHub Actions CI/CD complet.
- IA appliquee : integration LLM local (Ollama), prompt engineering, gestion de disponibilite.
- Paiements en ligne : Stripe Checkout, Connect, Webhooks, idempotence, commission splitting.

17.4 Perspectives d'Evolution

- Application mobile React Native : acces iOS/Android a toutes les fonctionnalites FreeNest.
- Architecture microservices : separation Auth Service, Payment Service, AI Service, Notification Service.
- IA multimodale : analyse des images de portfolio, reconnaissance de competences depuis un CV PDF.
- Video consultations integrees : entretiens client-freelancer en visio depuis la plateforme.
- Expansion internationale : multi-devises (EUR, MAD, GBP), traduction i18n, portails regionaux.
- Kubernetes : orchestration des containers en production haute disponibilite avec auto-scaling.
- Certifications freelancers : examen en ligne, badge verifie visible sur le profil.
- Blockchain : contrats intelligents pour les paiements escrow ultra-transparents (exploration).

17.5 Conclusion

FreeNest represente une realisation technique complete et ambitieuse, allant bien au-dela des standards d'un projet de fin d'etudes academique. C'est une application de marketplace full-stack de niveau production, integrant des technologies de pointe dans chaque couche : API REST robuste avec 100+ endpoints valides et securises, paiements en escrow avec comptabilite double-entree et garanties ACID, messagerie temps reel avec 11 evenements WebSocket sur canaux prives Reverb, intelligence artificielle locale (Ollama/Mistral 7B) sans exposition de donnees, infrastructure containerisee Docker et pipelines CI/CD GitHub Actions.

Ce projet demontre la capacite a concevoir, architechter et implementer un systeme logiciel complexe, multi-dimensionnel, en appliquant les meilleures pratiques de l'industrie 2026 : SOLID, securite defense-en-profondeur, tests automatises, deploiement reproductible. Il constitue une base solide pour une eventuelle mise en production et une alternative credible aux plateformes freelance etablies.

Annexes

A. Inventaire Complet des Endpoints API

Module	Endpoint (Methode + URI)	Acces
Auth	POST /auth/register POST /auth/login POST /auth/logout GET /auth/me PUT /auth/profile PUT /auth/password	Public
2FA	GET /two-factor/status POST /enable POST /confirm POST /disable GET /qr-code GET /recovery-codes	Authenticated

Module	Endpoint (Methode + URI)	Acces
KYC	GET /verify-identity/status POST /verify-identity GET /admin/kyc GET /admin/kyc/{id} POST approve	Auth
Jobs	GET /jobs GET /jobs/{id} POST /jobs POST /jobs/{id} DELETE /jobs/{id} POST /jobs/{id}/save GET	Public
Proposals	GET /jobs/{id}/proposals POST GET /my POST accept POST reject POST withdraw	Auth
Freelancer	GET /freelancers GET /freelancers/{user_id} GET /freelancer/{id} POST /freelancer/profile POST /sk	Auth
Contracts	GET /contracts (filtres) GET /{id} GET /my GET /active GET /completed GET /disputed POST complete cancel dispute	Auth
Milestones	GET /contracts/{id}/milestones POST GET /milestones/{id} PUT DELETE POST submit approve re	Auth
Files	GET POST /contracts/{id}/files GET /contracts/{id}/files/{id}/download POST version DELETE	Auth
Time	GET POST start stop /contracts/{id}/time GET /time/weekly	Auth
Extensions	GET POST /contracts/{id}/extensions POST /contract-extensions/{id}/respond	Auth
Analytics	GET /contracts/{id}/analytics GET /contracts/{id}/activity	Auth
PDF	GET /contracts/{id}/pdf /dispute-pdf	Auth
Payments	GET /wallet GET /overview POST /deposit POST /contracts/{id}/fund-escrow POST /milestones/{id}	Auth
Billing	GET /billing/subscription POST checkout swap cancel resume GET portal invoices GET GET /billing/	Auth
Chat	GET /conversations POST /start GET /{id} GET /messages POST /{id}/send GET /search POST /{id}/typ	Auth
AI	POST /ai/generate-proposal match-freelancers chat analyze-profile smart-search (throttle 20/min)	Auth
Catalog	GET /catalog GET /{slug} POST PUT DELETE POST save unsave GET /me/saved POST /orders/	Auth
Agencies	GET /agencies GET /{id} POST PUT DELETE GET members POST invitations POST invitations/	Auth
Talent	GET POST DELETE /saved-freelancers GET /check/{id} GET POST PUT DELETE /talent-lists POST	Auth
Tax	GET POST /tax-documents GET PDF /{id} GET POST approve reject /admin/tax-documents	Auth
Search	GET /search (throttle 60/min) GET /search GET /suggest (throttle 120/min)	Auth
Push	GET /push/vapid-public-key POST /push/subscribe DELETE /push/subscribe	Auth
Notifs	GET /notifications POST /read-all POST /{id}/read	Auth
Reviews	GET /reviews/freelancer/{id} POST /review POST /DELETE /{id}	Auth
Admin	GET /admin/dashboard GET /users POST /users/{id}/ban verify GET /analytics GET PUT /finance/s	Auth

B. Packages Backend (composer.json — production)

```
// backend-laravel/composer.json – require (production uniquement) "require": {
  "php": "^8.2", "laravel/framework": "^12.0", // Core Laravel "laravel/sanctum":
  "^4.3", // Bearer token auth "laravel/socialite": "^5.27", // OAuth2 Google
  "laravel/verb": "^1.10", // WebSocket server officiel "laravel/tinker":
  "^2.10.1", // REPL interactif "spatie/laravel-permission": "^6.25", // Roles et
  permissions "stripe/stripe-php": "^20.2", // Paiements Stripe
  "intervention/image": "^3.11", // Traitement images "tymon/jwt-auth": "^2.3", //
  JWT Auth alternatif "predis/predis": "^3.4" // Client Redis }
```

C. Packages Frontend (package.json — production)

```
// frontend-react/package.json – dependencies (production) "dependencies": {
  "react": "^19.2.6", "react-dom": "^19.2.6", "react-router-dom": "^7.15.1",
  "axios": "^1.16.1", "zustand": "^5.0.13", "framer-motion": "^12.39.0",
  "socket.io-client": "^4.8.3", "laravel-echo": "^2.3.4", "pusher-js": "^8.5.0",
  "recharts": "^3.8.1", "lucide-react": "^1.16.0", "@headlessui/react": "^2.2.10",
  "react-hot-toast": "^2.6.0", "date-fns": "^4.2.1" }
```

D. Glossaire Technique

Terme	Definition
Bearer Token	Token d'authentification HTTP transmis dans le header Authorization: Bearer <token>
Escrow	Mecanisme de tiers de confiance : les fonds sont bloques dans un compte intermediaire
Double-entree	Principe comptable : chaque transaction genere deux entrees opposees (debit + credit)
TOTP	Time-based One-Time Password (RFC 6238) : code a 6 chiffres genere par une application
VAPID	Voluntary Application Server Identification : protocole pour les Web Push Notifications
KYC	Know Your Customer : processus de verification de l'identite d'un utilisateur via des documents
HMAC	Hash-based Message Authentication Code : signature cryptographique verifiee par une cle
Idempotency Key	Cle unique attachee a une operation pour que les tentatives repetees ne produisent qu'un resultat
LLM	Large Language Model : modele de traitement du langage naturel a grande echelle
SPA	Single Page Application : application web ou la navigation se fait sans rechargement de page
Reverb	Serveur WebSocket officiel de Laravel, remplaçant Laravel Echo Server et compatible avec React
PSR-12	PHP Standard Recommendation 12 : standard de formatage du code PHP (espace de code)